Chapter 4: Developing Requirements

4.0 Domain Analysis

Domain analysis is learning about the business or technical field where software will operate. A domain expert has deep knowledge of that field.

- Understanding the domain speeds development, improves the solution, and helps anticipate extensions.
- A domain document includes a summary, glossary, general knowledge, customers and users, environment, current tasks and procedures, competing software, and similarities to other domains.
- Keep it accessible and maintainable, with useful diagrams and links.

4.1 Starting Points, Problem, and Scope

- Projects can start with a new opportunity or a problem in an existing system. Greenfield projects develop new systems.
- State the customer's difficulty or opportunity briefly before choosing a solution.
- Scope defines the work's boundary. List potential capabilities, then decide what belongs inside and outside it.
- Narrow an overly broad problem; revisit higher-level goals if the scope is too narrow.

4.2 What is a Requirement?

A requirement specifies what the proposed system must do or a constraint on its development. Together, requirements form a negotiated agreement among stakeholders.

4.2.0 Writing a good requirement

- Identify the system, required action or outcome, relevant conditions, and a measurable success criterion.
- Use complete sentences and consistent terminology. Use shall for mandatory behaviour and make optional status explicit.
- Specify the outcome without unnecessarily dictating implementation. Confirming a purchase need not require Microsoft Outlook unless that is a genuine platform constraint.
- Slide example: the online banking system shall allow an Internet user to access her current account balance in less than five seconds.
- For testing, clarify the response-time measurement's start, end, and workload conditions.

4.2.1 Pitfalls

- Avoid vague terms such as quickly, user-friendly, flexible, minimal, or usually unless measurements define them.
- Avoid speculative wishes, incomplete sentences, and assumptions about what users probably want.
- Avoid impossible promises such as handling every conceivable failure or running on every platform.
- State obligations for the system. Describing what a user happens to do does not adequately specify system behaviour.

4.3 Types of Requirements

- Functional requirements describe inputs, outputs, stored data, computations, and required timing or synchronization of actions.
- Quality requirements specify measurable response time, throughput, resource use, reliability, availability, recovery, maintainability, or reusability.
- Platform requirements constrain the environment and technology.
- Process requirements constrain development activities, methods, or the project plan.
- Retrieving a balance is functional behaviour; the response-time limit is a related quality requirement.

4.4 Use Cases

A use case describes interactions through which an actor achieves a goal with the system. Actors are roles played by users or external participants.

- Focus on observable interactions, avoiding internal algorithms and unnecessary screen-design details.
- A use-case model contains use cases and optionally a description or diagram of their relationships.

4.4.0 Description format

- Name: short task description. Actors: participating roles. Goals: what those roles seek to achieve.
- Preconditions: what must already be true. Summary: brief overview. Related use cases: dependencies or specializations.

- Steps: actor actions and system responses, commonly in two columns. Include alternative and exceptional paths.
- Postconditions: the state after completion. Distinguish success and failure where necessary.

4.4.1 Relationships

- Include factors shared behaviour into another use case. The dependency points from the including case to the included case.
- Extend adds conditional or optional behaviour at an extension point. The dependency points from the extending case to the base case.
- Generalization captures common behaviour with specialized variants. The arrow points toward the more general case or actor.
- Inclusion reuses a lower-level task; extension keeps conditional behaviour separate from the basic flow.

4.4.2 Scenarios and priorities

- A scenario is a concrete occurrence of a use case with specific actors, data, and events.
- Prioritize central use cases, risky functions, and work with high commercial or stakeholder value.
- Use cases help define scope, plan development, validate requirements, derive tests, and organize user manuals.
- Agile user stories are typically shorter and less formal statements of needs. Discussion and acceptance criteria supply implementation detail.

4.4.3 Study example: registering for a course

- Actor: student. Goal: obtain a course place. Preconditions: authenticated student and open registration period.
- The student requests a course. The system displays details and availability. The student requests enrolment. The system checks eligibility and capacity, records the enrolment, and confirms it.
- If capacity or prerequisites prevent enrolment, the system explains the rejection and does not record successful enrolment.
- Success postcondition: an enrolment links the student to the course offering. This is an explanatory study example.

4.5 Gathering and Analysing Requirements

- Observation: review documents and watch users work; ask them to explain decisions and exceptions.
- Interviews: ask about details, future goals, alternatives, other information sources, and helpful diagrams.
- Brainstorming: use a moderator and a clear trigger question. Joint Application Development uses intensive collaborative sessions.
- Prototypes: paper sketches or UI mock-ups help users react to proposed interactions; they may omit computation, databases, and integration.
- Use-case analysis identifies actors and the tasks each needs to accomplish.
- Repeat these techniques as understanding improves. More detail is needed for complex systems, external interfaces, technical domains, and costly consequences of error.

4.6 Reviewing Requirements

- Each requirement should be valuable, relevant, clear, unambiguous, consistent, feasible, verifiable, and uniquely identifiable.
- Avoid unnecessary design restrictions and check that the resulting system can meet quality goals.
- Review the entire set for completeness, organization, consistency, and stakeholder agreement.
- Traceability links requirements to their origins, design, implementation, and tests, supporting change-impact analysis.

4.6.0 Ambiguity example

The slide's interest-calculation example asks for edits to one of three fields to compute the other two. One value generally cannot determine two unknowns. Specify which additional value remains fixed and how each edit is interpreted.

4.7 Managing Changing Requirements

- Requirements change with business processes, technology, and improved understanding. Analysis continues throughout development.
- Assess each change's benefit, effort, design effects, schedule impact, and affected tests.
- Keep clients and users involved. Small presentation changes may be inexpensive; structural changes can require significant redesign.

- Distinguish corrections from enhancements that expand scope. Update specifications and traceability after accepting a change.